ΟΙ ΒΑΣΙΚΕΣ ΕΠΑΝΑΛΗΠΤΙΚΕΣ ΔΟΜΕΣ

ΚΑΙ ΟΙ ΡΟΕΣ

Μ.Ι.Σ. 2025-2026

Η ΕΝΤΟΛΗ IF

Σε όλες τις γλώσσες προγραμματισμού η βασική εντολή για τον έλεγχο της ροής του προγράμματος, είναι η εντολή **if**

Για την επίλυση πολύπλοκων προβλημάτων πολλές φορές καθίσταται αναγκαίο κάποιες εντολές του προγράμματος να μην εκτελούνται με τη σειρά που είναι γραμμένες. Η **δομή ελέγχου** είναι η δομή που χρησιμοποιείται για την επιλογή μιας επεξεργασίας μεταξύ δύο ή περισσότερων διαφορετικών διαδικασιών ανάλογα με την τιμή μιας λογικής συνθήκης.

Η δομή ελέγχου χρησιμοποιείται με δύο εκδοχές,

απλή εντολή έλεγχου
σύνθετη εντολή ελέγχου.

Η σύνταξη της απλής εντολής είναι:

```
if (συνθήκη ελέγχου):
        εντολή 1
        εντολή 2
        …
        εντολή ν
```

Η σύνταξη λειτουργεί και ως σύνθετη εντολή ελέγχου. Με την εντολή αυτή επιλέγεται μία από τις δύο επεξεργασίες που μπορούν να γίνουν, ανάλογα με την τιμή μιας συνθήκης ελέγχου.

```
if (συνθήκη ελέγχου) :
        ομάδα εντολών 1
else :
```

Μ.Ι.Σ. 2025-2026

ομάδα εντολών 2

Όταν οι εναλλακτικές περιπτώσεις είναι περισσότερες από δύο, τότε μπορούν να χρησιμοποιηθούν πολλές εντολές **if** κάθε μία μέσα στην άλλη, που ονομάζονται **εμφωλιασμένες *δομές ελέγχου***,

```
if (συνθήκη ελέγχου 1) :
        ομάδα εντολών 1
else :
        if (συνθήκη ελέγχου 2):
                ομάδα εντολών 2
                …
        else:
                ομάδα εντολών ν
```

Όταν κάποια από τις συνθήκες ελέγχου είναι αληθής, εκτελούνται οι εντολές που βρίσκονται στο αντίστοιχο τμήμα. Κάθε τμήμα της δομής ελέγχου μπορεί να περιέχει οποιεσδήποτε εντολές. Συνεπώς μπορεί να περιλαμβάνει και άλλες δομές ελέγχου.

Εάν μέσα σε μια εντολή **if** ή στο **else** της, δεν θέλουμε να εκτελεστεί τίποτα, μπορούμε να χρησιμοποιήσουμε την εντολή **pass** στην θέση εντολής.

ΠΡΟΣΟΧΗ, στην PYTHON για να λειτουργήσει σωστά η δομή των **if** θα πρέπει να τηρούμε τον κανόνα, <u>ότι οι εντολές του **if** θα πρέπει να είναι μετατοπισμένες όλες προς τα δεξιά με τέσσερα κενά</u>. Η έλλειψη των 4 κενών ή ύπαρξη επιπλέον κενών μπορεί να οδηγήσει σε λάθος συμπεριφορά των **if**

Μ.Ι.Σ. 2025-2026

Ειδικά για την **if** υπάρχει και η δυνατότητα εκτέλεσης με πολλούς κλάδους εκτέλεσης. Κάθε συνθήκη ελέγχεται με τη σειρά, εάν η πρώτη είναι ψευδής, ελέγχεται η δεύτερη, εάν η δεύτερη είναι ψευδής, ελέγχεται η τρίτη, κ.λπ. Η εντολή προσπαθεί να προσομοιώσει την ευκολία της εντολής **switch** της κλασσικής C

η PYTHON δεν έχει την εντολή **switch**

if (συνθήκη ελέγχου 1) ***:***

ομάδα εντολών 1

elif (συνθήκη ελέγχου 2)***:***

ομάδα εντολών 2

else*:*

ομάδα εντολών ν

ΠΡΟΣΟΧΗ, εάν κάποια συνθήκη είναι αληθής, τότε ο αντίστοιχος κλάδος με όλες τις εντολές του εκτελείται και η εντολή **if** ολοκληρώνεται, ακόμη και αν ακολουθούν και άλλες συνθήκες που μπορεί να αληθεύουν.

η PYTHON δεν έχει την εντολή **switch**

Μ.Ε. 2025-2026

Η ΕΝΤΟΛΗ for

Η εντολή **for** στην PYTHON μπορεί να περιέχει τρείς ενέργειες που θα γίνουν προκειμένου να εκτελεστεί ένα συγκεκριμένο κομμάτι κώδικα.

Μια μεταβλητή (μετρητής) πρέπει να πάρει μια αρχική τιμή, για να συγκριθεί στην συνέχεια με κάποιο όριο κάθε φορά που η δομή εκτελείται.

Συνήθως συντάσσουμε την εντολή **for** με την «εργοστασιακή» συνάρτηση **range** ώστε να παρέχουμε αυτόματα βηματισμό στην εντολή.

Δηλαδή το,

```
for i in range(4,12)
```

είναι αντίστοιχο με την σύνταξη,

```
for i in [4, 5, 6, 7,8,9,10,11]
```

Η ανάθεση αρχικών τιμών εκτελείται μόνο μία φορά πριν εκτελεστεί (στην αρχή) κάποια από τις εντολές του βρόχου.

Αν η συνθήκη ελέγχου είναι αληθής (ή μη μηδενική), ο βρόχος εκτελείται μία φορά. Μετά υπολογίζεται η έκφραση ανανέωσης και η συνθήκη ελέγχου ελέγχεται ακόμα μια φορά.

Στην **for** η απόφαση για την εκτέλεση του βρόχου, γίνεται πριν την εκτέλεση της for. Έτσι, το **for** μπορεί να μην εκτελεστεί ποτέ.

Μ.Ι.Σ. 2025-2026

Το μέρος του κώδικα (προς εκτέλεση) μπορεί να αποτελείται από μια απλή εντολή ή από πολλές εντολές.

```
for  w_i  in  range(10):
        print (w_i)
```

οι εντολές του βρόγχου πρέπει να είναι μέσα στην ίδια εσοχή 4 κενών

η ανωτέρω σύνταξη θα εκτελεστεί 10 φορές, από το 0 (κατάλοιπο της C) μέχρι και το 9, η σύνταξη καθώς δεν περιέχει κλιμάκωση του **w_i** θα λειτουργήσει με τον προαιρετικό (και εξ'ορισμού) βηματισμό για +1

Μια εντολή **for** μπορεί να έχει και έναν προαιρετικό όρο **else**

```
for w_i in range(1, 5):
        print (w_i)
else:
        print ('Το loop τέλειωσε')
```

Μ.Ι.Σ. 2025-2026

Η εντολή **for** μπορεί επιπλέον να οριστεί για συγκεκριμένο σημείο εκκίνησης από το **w_i** και για συγκεκριμένο βηματισμό,

```
for  w_i  in  range(1, 10, 2):
        print (w_i)
```

η ανωτέρω σύνταξη θα εκτελεστεί 5 φορές, από το 1 μέχρι και το 10, και με βηματισμό για +2, οπότε θα τυπώσει

```
1 3 5 7 9
```

στην PYTHON η σύνταξη μπορεί να περιέχει τρείς ενέργειες που θα γίνουν προκειμένου να εκτελεστεί ένα συγκεκριμένο κομμάτι κώδικα.

Μ.Ι.Σ. 2025-2026

Οι παρενθέσεις που ακολουθούν την **range** περιέχουν τρείς εκφράσεις χωρισμένες με κόμμα.

- Η πρώτη έκφραση δίνει την αρχική τιμή και εκτελείται μια φορά όταν ο βρόχος **for** ξεκινάει
- Η δεύτερη έκφραση είναι μια συνθήκη ελέγχου, και υπολογίζεται πριν από κάθε εκτέλεση του βρόχου. Όταν η συνθήκη γίνει ψευδής, ο βρόχος τερματίζει.
- Η τρίτη έκφραση υπολογίζεται στο τέλος κάθε βρόχου (αυξάνει βηματικά το **w_i**). Η τρίτη έκφραση μπορεί να υπολογίζεται και με αρνητικό βηματισμό (όπως και σε όλες τις γλώσσες).

Ειδικότερα για την **range** θα πρέπει να λάβουμε υπόψη, ότι μπορούμε να συντάξουμε,

- **range(mynum)** δημιουργεί ακολουθία αριθμών από το **0** ως την προηγούμενη ακέραια τιμή της **mynum**
- **range(myfrom, myto)** δημιουργεί ακολουθία αριθμών από την τιμή της **myfrom** μέχρι την προηγούμενη ακέραια τιμή της **myto**
- **range(myfrom, myto, mystep)** δημιουργεί ακολουθία αριθμών από την τιμή της **myfrom** μέχρι την προηγούμενη ακέραια τιμή της **myto** με βήμα **mystep**

Μπορούμε να έχουμε έναν βρόγχο **for** μέσα σε έναν άλλο βρόγχο **for** (**εμφωλιασμένος βρόγχος**).

Μ.Ι.Σ. 2025-2026

Η ΕΝΤΟΛΗ while

Σημαντική εντολή που επαναλαμβάνει κομμάτι κώδικα επάνω σε έναν βρόγχο.

Ενας βρόχος (κόμβος με κώδικα) που επαναλαμβάνεται όσο η *συνθήκη* είναι ΑΛΗΘΗΣ (δηλαδή ≠ 0). Όταν η *συνθήκη* γίνει ΨΕΥΔΗΣ (δηλαδή = 0), τότε ο έλεγχος του προγράμματος πάει στην γραμμή μετά το βρόγχο.

Οι εντολές του βρόγχου θα πρέπει να είναι μέσα στην ίδια εσοχή των 4 κενών.

```
w_i = 0

while ( w_i < 10 ):
        print w_i
        w_i += w_i
```

η ανωτέρω σύνταξη θα εκτελεστεί 10 φορές, από το 0 μέχρι και το 9

την κλιμάκωση του **w_i** θα πρέπει να την καθορίσουμε προγραμματιστικά καθώς δεν επηρεάζεται αυτόματα

Οι βρόχοι **while** ελέγχουν την συνθήκη στην αρχή του βρόχου, και αυτό σημαίνει ότι ανάλογα με την τιμή της συνθήκης ο κώδικας του βρόχου <u>μπορεί να μην εκτελεστεί ποτέ</u>.

Μπορούμε να έχουμε έναν βρόγχο **while** μέσα σε έναν άλλο βρόγχο **while** (**εμφωλιασμένος βρόγχος**).

Μ.Π.Σ. 2025-2026

Μια εντολή **while** μπορεί να έχει και έναν προαιρετικό όρο **else**

```
w_i = 0

while ( w_i < 10 ):
        print w_i
        w_i += w_i
else:
        print ('ektos')
```

Μ.Ι.Σ. 2025-2026

Η ΕΝΤΟΛΗ break

Η **break** μπορεί να χρησιμοποιηθεί με τις επαναληπτικές εντολές **for, while**

Έχει σαν αποτέλεσμα τη διακοπή εκτέλεσης της επαναληπτικής δομής και τη μετάβαση του ελέγχου του προγράμματος στην εντολή που βρίσκεται αμέσως μετά το βρόχο.

Η ΕΝΤΟΛΗ continue

Η εντολή **continue** μπορεί να χρησιμοποιηθεί με τις επαναληπτικές δομές **for, while**

Η **continue** (όπως και η **break**) διακόπτει τη ροή ενός προγράμματος.

Όμως αντί να τερματίσει ολόκληρο το βρόχο, η **continue** έχει ως αποτέλεσμα τη μη εκτέλεση του υπόλοιπου τμήματος του βρόχου και την επανάληψή του από την αρχή.

Μ.Ι.Σ. 2025-2026